ARES

Autonomous Reliability Engineering System

Complete Project Documentation — Phase 1 to Phase 2 Roadmap

AI-Driven Detection • Self-Healing Infrastructure • Production-Grade DevOps

Powered by Helios ML Research • Accepted Conference Paper

Table of Contents

1. What Is This Document?
2. Understanding ARES — The Big Picture
3. The Research Foundation — Helios
4. System Architecture — How Everything Fits Together
5. Phase 1 — What Has Been Built (Complete)
6. The Technology Stack — Every Tool Explained
7. How the Application Works — Full User Flow
8. The Database Layer
9. Deployment — How the App Runs on the Internet
10. Phase 2 — What Will Be Built Next
11. The ARES Control Plane — The Reliability Brain
12. Helios Integration — The ML Engine in ARES
13. The Self-Healing Demo — Exactly What Will Happen
14. Team Structure and Responsibilities
15. Glossary — Every Technical Term Explained

1. What Is This Document?

This document is a complete, end-to-end technical documentation of the ARES project. It is written so that anyone — whether a student, professor, engineer, or evaluator — can read it and fully understand what the project is, why it was built, how it works technically, what has already been done, and exactly what will be built next.

No prior knowledge of cloud computing, DevOps, or machine learning is assumed. Every technical concept is explained from scratch, with real-world analogies wherever possible. By the end of this document, you will understand:

- The full system architecture of ARES
- Why the project is technically advanced and research-backed
- Every technology used, what it does, and why it was chosen
- The exact state of the project today (Phase 1 — complete and deployed)
- The detailed plan for Phase 2 — the actual ARES intelligence layer
- How Helios, an accepted conference research paper, forms the ML brain of ARES

2. Understanding ARES — The Big Picture

2.1 The Problem ARES Solves

Imagine a large e-commerce website — thousands of users shopping at the same time. Behind the scenes, this website is not one program. It is made of many small programs (called microservices) running simultaneously: one handles user login, another handles product listings, another manages the shopping cart, another processes orders.

Now imagine one of these programs crashes at 3 AM. A traditional system would:

1. Detect that the service is down
2. Send an alert email or SMS to an engineer
3. Wait for the engineer to wake up, log in, diagnose the problem, and fix it manually
4. Meanwhile, the website shows errors to users for 30–60 minutes

This is the problem. Human response time creates downtime. Downtime costs money, users, and reputation.

ARES Solution	ARES eliminates human response time entirely. It detects the failure, decides the correct fix, executes the recovery action, and logs the event — all automatically, within seconds.

2.2 The ARES Control Loop

ARES operates on a principle from computer science called the MAPE-K loop — Monitor, Analyze, Plan, Execute, with a shared Knowledge base. This is the foundation of all autonomous computing systems.

Step	What ARES Does	Example
Monitor	Continuously collects system metrics every few seconds	CPU at 87%, response time 2400ms
Analyze	ML Engine checks if metrics are abnormal	Isolation Forest scores this as anomalous
Plan	Decision Engine chooses the right action	Order Service should be restarted
Execute	Healing Agent performs the action automatically	docker restart order-service
Knowledge	Everything is logged for learning and audit	Event stored in dashboard timeline

2.3 The Two Layers of ARES

ARES is built in two distinct layers. Understanding this separation is the key to understanding the whole project.

Layer	Name	What It Is	Status
Layer 1	Application Data Plane	The real e-commerce website with microservices — the system that can fail	Complete (Phase 1)
Layer 2	ARES Control Plane	The AI reliability brain that monitors, detects, and heals Layer 1	In Progress (Phase 2)

A useful analogy: Layer 1 is a patient in a hospital. Layer 2 is the ICU monitoring system and automated medical response team. The patient is real — they have real activity, real failures. The monitoring system watches them 24/7 and acts the moment something goes wrong.

3. The Research Foundation — Helios

3.1 What Is Helios?

Helios is a cloud-native AI anomaly detection system built by Nitanshu Tak as part of a research project. The research paper based on Helios has been accepted at an academic conference, making it a validated, peer-reviewed contribution to the field of DevSecOps and cloud intelligence.

Helios focuses specifically on using machine learning to detect anomalies in cloud infrastructure metrics. It was built on AWS and uses scikit-learn ML models trained to recognize five types of cloud system problems:

- CPU Utilization spikes — detecting when a server is being overloaded
- Network Traffic anomalies — detecting abnormal volumes of requests
- Unauthorized Login Attempts — detecting brute-force or suspicious access patterns
- Content Injection — detecting malicious or spam text being submitted to the system
- Storage Growth anomalies — detecting sudden unexpected increases in disk usage

3.2 How Helios Works Technically

Helios uses a Flask API server deployed on AWS EC2. The ML models are trained in Google Colab using Python and scikit-learn, then exported as .pkl (pickle) files and stored in AWS S3. When metrics are submitted to the API, the correct model is loaded and produces an anomaly prediction.

```
POST /predict/cpu   {"value": 95}    ->  {"anomaly": true,  "score": 0.87}

POST /predict/network  {"value": 600}  ->  {"anomaly": false, "score": 0.12}
```

3.3 Why Helios Is the Brain of ARES

ARES needs an ML engine that can look at CPU usage, memory consumption, and response time data — and decide whether the system is behaving normally or heading toward a failure. This is exactly what Helios was researched, built, and validated to do.

The connection is not forced or superficial. Helios literally does what ARES's ML Engine needs to do. ARES extends Helios in one critical direction: while Helios detects and alerts, ARES detects, decides, and heals. The detection logic is Helios. The healing action is ARES's contribution on top.

Academic Value	Because Helios is an accepted conference paper, ARES inherits its research credibility. ARES can be described as an applied implementation and extension of the Helios anomaly detection research — from standalone detection to fully autonomous cloud reliability engineering.

Capability	Helios	ARES (with Helios)
Anomaly Detection	Yes — ML models for 5 metric types	Yes — integrated as ARES ML Engine
Real-time Alerting	Yes — via AWS SNS email	Yes — displayed on ARES Dashboard
Automated Recovery	No — human must act	Yes — Healing Agent acts automatically
Microservices Awareness	No — generic cloud metrics	Yes — per-service monitoring and healing
Research Backing	Accepted conference paper	Extends the paper into full implementation

4. System Architecture — How Everything Fits Together

4.1 The Three Planes

The ARES architecture is organized into three planes, visible in the architecture diagram:

Plane	Components	Purpose
User & Load Generation	Next.js Frontend, Locust Load Generator	The user-facing website and traffic simulation tools
Application Data Plane	API Gateway, 4 Microservices, PostgreSQL, Redis, Docker Network	The actual e-commerce application running in containers
ARES Control Plane	Monitoring Agent, ML Engine, Decision Logic, Healing Agent, Dashboard	The autonomous reliability intelligence system

4.2 The Request Flow

Here is the complete path a user request takes through the system:

5. User opens the website in their browser (Next.js frontend loads from Vercel)
6. User clicks 'Add to Cart' — the frontend sends an API request to the backend
7. The request hits the API Gateway, which routes it to the correct microservice
8. The Cart Service processes the request and updates the PostgreSQL database
9. The response travels back through the gateway to the frontend
10. Meanwhile, ARES Monitoring Agent is continuously scraping metrics from all services
11. If any metric looks abnormal, the ML Engine flags it and the Healing Agent responds

4.3 Why Microservices Architecture?

The most important architectural decision in ARES is splitting the backend into independent microservices instead of building one single backend application. This decision has a very specific reason in the context of ARES.

If the backend were a single program, when it fails — everything fails at once. There is nothing interesting to monitor or heal. But with microservices, the User Service can be running fine while the Order Service is crashing under load. This creates realistic, targeted failure scenarios that ARES can detect and fix at the individual service level.

Key Insight	Microservices are not just an architecture choice — they are a requirement for building a meaningful self-healing system. Without independent services that can fail independently, autonomous reliability engineering has nothing to work with.

5. Phase 1 — What Has Been Built (Complete)

5.1 Overview

Phase 1 is fully complete. The e-commerce application is live, deployed on the internet, and fully functional. Every part of the user-facing application works end-to-end with real data.

Live URL	https://letslumiere.vercel.app — The application is publicly accessible. You can register an account, browse products, add items to the cart, place an order, and view your order history in real time.

5.2 What Was Built in Phase 1

5.2.1 The Frontend Application

The frontend is a complete, production-quality e-commerce website built with Next.js and TypeScript. It was designed and built by the team's frontend developer Aarohi Jaiswal and features a premium, modern visual design.

The frontend includes the following pages and features:

- **Landing Page —** The home page of the website with brand identity, featured products, and navigation
- **Product Catalog —** A browsable grid of all products fetched in real-time from the backend
- **User Authentication —** A complete Register and Login system with JWT token-based sessions
- **Shopping Cart —** Add items, view cart contents, remove items, all persisted in the database
- **Checkout & Orders —** Place real orders and view the full order history with status tracking
- **Responsive Design —** Works correctly on desktop, tablet, and mobile screens

5.2.2 The Backend Microservices

The backend is split into four independent services, each responsible for one domain of the application. Every service is a Node.js Express application connected to its own PostgreSQL database on Neon (cloud-hosted PostgreSQL).

Service	Port	Responsibility	Key Endpoints	Deployed URL
User Service	3001	Authentication and user profile management	POST /users/register, POST /users/login, GET	user-service-ekmh.onrender.com

Service	Port	Responsibility	Key Endpoints	Deployed URL
			/users/profile, GET /health	
Product Service	3002	Product catalog and inventory	GET /products, GET /products/:id, POST /products, GET /health	product-service-t6mo.onrender.com
Cart Service	3003	Shopping cart operations per user	GET /cart, POST /cart/add, DELETE /cart/remove, GET /health	cart-service-thp5.onrender.com
Order Service	3004	Order placement and tracking	POST /order, GET /orders, GET /orders/:id, GET /health	order-service-s3gt.onrender.com

Every service includes a /health endpoint. This is not just a formality — it is the entry point for the ARES Health Agent in Phase 2. When the Health Agent calls GET /health on each service and receives no response, it knows that service has failed and triggers the healing process.

5.2.3 Database Architecture

The application uses PostgreSQL as its primary relational database. Each microservice has its own isolated database — this is called the Database-per-Service pattern, and it is a core principle of microservices architecture.

Database	Service	Key Tables	Hosted On
ecommerce_users	User Service	users (id, name, email, password_hash, created_at)	Neon (AWS ap-southeast-1)
ecommerce_products	Product Service	products (id, name, description, price, stock, image_url)	Neon (AWS ap-southeast-1)
ecommerce_cart	Cart Service	cart_items (id, user_id, product_id, quantity)	Neon (AWS ap-southeast-1)
ecommerce_orders	Order Service	orders, order_items (full order records with status)	Neon (AWS ap-southeast-1)

The reason for separate databases (rather than one shared database) is fault isolation. If the Order Service database experiences a problem, the User Service and Product Service are completely unaffected. Users can still browse products and log in even if order placement is temporarily unavailable.

5.2.4 Authentication and Security

The application uses JWT (JSON Web Token) authentication. Here is how it works in plain terms:

12. User registers with email and password. The password is never stored as plain text — it is hashed using bcrypt before being saved to the database.
13. User logs in. The backend verifies the password hash and, if correct, generates a JWT — a small encrypted token containing the user's ID.
14. This token is stored in the browser's localStorage. Every subsequent request to protected endpoints (cart, orders) includes this token in the Authorization header.
15. Each service verifies the token before processing the request. If the token is missing or invalid, the request is rejected with a 401 Unauthorized response.

6. The Technology Stack — Every Tool Explained

6.1 Frontend Technologies

Technology	What It Is	Why ARES Uses It
Next.js	A React framework for building web applications with server-side rendering	Industry standard for modern web apps. Provides routing, optimization, and deployment compatibility with Vercel.
React	A JavaScript library for building user interfaces with reusable components	Allows the UI to update in real-time without full page reloads — essential for a live dashboard.
TypeScript	JavaScript with strict type checking	Catches bugs at development time, not runtime. Makes the codebase more reliable and maintainable.
Tailwind CSS	A utility-first CSS framework for styling	Enables rapid, consistent, professional UI design without writing custom CSS files.
Vercel	A cloud platform optimized for Next.js deployments	One-click deployment, global CDN, automatic HTTPS. The frontend is live within minutes of a code push.

6.2 Backend Technologies

Technology	What It Is	Why ARES Uses It
Node.js	A JavaScript runtime that runs server-side code	Allows using JavaScript on the backend. Fast, event-driven architecture ideal for API services.
Express.js	A minimal web framework for Node.js	Simple and flexible. Makes it easy to define API routes and middleware for each microservice.
PostgreSQL	A powerful, open-source relational database	ACID-compliant transactions are critical for e-commerce (you cannot lose an order). Highly reliable.
Neon	A serverless cloud-hosted PostgreSQL provider	Free tier, automatic scaling, no server management. Connection strings work the same as local PostgreSQL.
JWT (jsonwebtoken)	A library for creating and verifying JSON Web Tokens	Stateless authentication — services can verify user identity without calling the User Service on every request.
bcryptjs	A library for hashing passwords	Passwords are never stored as plain text. Even if the database is compromised, actual passwords cannot be recovered.

Technology	What It Is	Why ARES Uses It
Render	A cloud platform for deploying backend services	Free tier for web services. The 4 backend services are deployed here and accessible from anywhere on the internet.

6.3 Phase 2 Technologies (Coming Next)

Technology	What It Is	Role in ARES
Docker	A containerization platform	Each microservice runs in its own isolated container. This makes it possible to kill and restart individual services programmatically.
Prometheus	An open-source metrics collection and monitoring system	Scrapes /metrics endpoints from each service every 15 seconds. Stores time-series data of CPU, memory, response time.
Grafana	A data visualization platform	Connects to Prometheus and displays real-time graphs of all system metrics. The visual monitoring dashboard.
Python (scikit-learn)	A machine learning library for Python	Trains and runs the Isolation Forest anomaly detection model — directly from Helios research.
Flask	A lightweight Python web framework	The ML Engine is wrapped in a Flask API (from Helios). ARES agents call this API to get anomaly scores.
AWS EC2	Amazon's cloud virtual machine service	Hosts the Dockerized microservices and ARES agents in a production cloud environment.

7. How the Application Works — Full User Flow

7.1 Step-by-Step User Journey

This section walks through exactly what happens — technically — when a user interacts with the ARES application.

Step 1: User Opens the Website

The user visits https://letslumiere.vercel.app. Vercel serves the Next.js application from its global CDN. The browser downloads the page and JavaScript. The frontend starts running in the browser.

Step 2: User Registers an Account

The user fills in their name, email, and password and submits the registration form. The frontend sends a POST request to the User Service:

```
POST https://user-service-ekmh.onrender.com/users/register

Body: { "name": "Rahul Sharma", "email": "rahul@email.com", "password":
"mypassword" }
```

The User Service receives this request. It hashes the password using bcrypt (producing a long encrypted string that cannot be reversed), then saves the user record to the ecommerce_users database on Neon. It responds with a JWT token. The frontend stores this token in localStorage.

Step 3: User Browses Products

The frontend sends a GET request to the Product Service:

```
GET https://product-service-t6mo.onrender.com/products
```

The Product Service queries the ecommerce_products database and returns the full list of products as JSON. The frontend renders each product as a card on the screen.

Step 4: User Adds Item to Cart

The frontend sends a POST request to the Cart Service with the JWT token in the header:

```
POST https://cart-service-thp5.onrender.com/cart/add

Headers: { Authorization: Bearer <jwt_token> }

Body: { "product_id": "42", "quantity": 2 }
```

The Cart Service verifies the JWT token, extracts the user ID, and saves the cart item to the ecommerce_cart database linked to that user.

Step 5: User Places an Order

The frontend calls the Order Service, which processes the entire order in a single database transaction — meaning either all of it succeeds or none of it does. This prevents scenarios where payment is taken but no order record is created.

```
POST https://order-service-s3gt.onrender.com/order
```

The Order Service records the order, assigns it a status of 'processing', and returns the order ID to the frontend. The user sees a confirmation screen with their order number.

8. The Database Layer

8.1 Why PostgreSQL?

PostgreSQL was chosen over other database options (like MongoDB or MySQL) for a specific reason: ACID compliance. ACID stands for Atomicity, Consistency, Isolation, and Durability — the four properties that guarantee database transactions are processed reliably.

For an e-commerce system, this is non-negotiable. When a user places an order, the database must either fully complete the transaction (deduct stock, create order, confirm payment) or fully roll it back (as if the transaction never happened). There can be no partial states where the user was charged but the order was not created.

8.2 Database-per-Service Pattern

One of the most important architectural decisions in the database layer is that each microservice has its own completely separate database. This is not just a best practice — it is a requirement for the ARES reliability system.

Consider what happens if all services share one database and that database goes down. Everything fails at once — there is no service isolation. But with separate databases, the Product Service can be browsed even if the Order Service database is having issues. This is exactly the kind of targeted, isolated failure that ARES is designed to detect and heal.

8.3 Redis (Phase 2)

In Phase 2, Redis will be added as an in-memory data store. Redis serves two purposes in ARES: session caching (making repeated requests faster by storing frequently-accessed data in memory rather than querying PostgreSQL every time) and as a temporary store for real-time metrics that need to be processed by the ML Engine. Redis is extremely fast — it can handle hundreds of thousands of operations per second because it stores everything in RAM rather than on disk.

9. Deployment — How the App Runs on the Internet

9.1 The Deployment Architecture

The Phase 1 application is deployed across three cloud platforms, each chosen for the specific role it plays best:

Component	Platform	Why This Platform
Frontend (Next.js)	Vercel	Vercel is built by the creators of Next.js. One-click deployment, global CDN, automatic HTTPS, and every git push triggers automatic redeployment.
Backend Services (4 microservices)	Render	Free tier cloud hosting for Node.js services. Each service has its own URL and runs independently. Services auto-restart if they crash.
Database (PostgreSQL)	Neon	Serverless PostgreSQL with a generous free tier. Connection strings are standard PostgreSQL — no code changes needed compared to local development.

9.2 The CI/CD Pipeline

CI/CD stands for Continuous Integration / Continuous Deployment. It means that every time code is pushed to the GitHub repository, the system automatically builds, tests, and deploys the updated application — no manual deployment steps required.

The current CI/CD flow for the frontend works as follows:

16. Developer pushes code to the main branch of the GitHub repository
17. Vercel detects the push via a GitHub webhook
18. Vercel automatically pulls the latest code and runs the Next.js build process
19. If the build succeeds, the new version is deployed globally within 1-2 minutes
20. If the build fails, the previous working version stays live — zero downtime

In Phase 2, a full GitHub Actions CI/CD pipeline will be added for the backend services, including building Docker images, pushing them to Amazon ECR (Elastic Container Registry), and deploying them to AWS EC2 automatically.

10. Phase 2 — What Will Be Built Next

10.1 Phase 2 Overview

Phase 2 is the construction of the ARES Control Plane — the actual reliability intelligence system. Phase 1 built the patient. Phase 2 builds the doctor.

Phase 2 consists of five sub-phases, each building on top of the previous one:

Sub-Phase	Name	What Gets Built	Output
2A	Containerization	Docker containers for all 4 microservices + docker-compose orchestration	Services run in containers that can be killed and restarted programmatically
2B	Health & Healing Agents	Python scripts that check /health endpoints and call docker restart when services fail	First working self-healing demo — kill a container, watch it auto-restart
2C	Observability Stack	Prometheus metrics collection + Grafana visualization dashboard	Real-time graphs of CPU, memory, response time, error rate for all services
2D	ML Anomaly Detection	Helios ML models integrated as ARES ML Engine, fed by Prometheus data	System predicts failures before they happen, not just after
2E	ARES Dashboard	Frontend control center showing service health, ML alerts, recovery events	Full visual demonstration of the autonomous reliability system

10.2 Why This Order?

The sub-phases are ordered by dependency, not by importance. You cannot have a Healing Agent without Docker (because the agent calls docker restart). You cannot feed data to the ML Engine without Prometheus collecting it first. Each layer is the foundation for the next.

11. The ARES Control Plane — The Reliability Brain

11.1 The Five Agents

The ARES Control Plane is built around five autonomous agents. Each agent is an independent background process (running in its own Docker container) with a single, focused responsibility. They run continuously, they never sleep, and they communicate with a shared ARES backend.

Agent	Language	What It Does	How Often It Runs
Monitoring Agent	Python	Calls the Prometheus API and collects CPU usage, memory consumption, API response time, and error rate for every service	Every 15 seconds
Health Agent	Python	Sends a GET /health request to each of the 4 microservices. If no response within 3 seconds, the service is marked as DOWN	Every 10 seconds
Log Agent	Python	Reads log files from each service container and scans for error patterns, exception messages, and unusual log frequencies	Continuously (tail -f style)
Healing Agent	Python	When triggered by Health Agent or ML Engine, executes recovery commands via the Docker Engine API. Logs every action.	On-demand (event-driven)
Load Agent	Python (Locust)	Generates synthetic traffic to the e-commerce application — simulating real user behavior at scale for stress testing	On-demand (for demos)

11.2 The Decision Engine

The Decision Engine is the logic layer that sits between anomaly detection and healing action. It answers the question: given that we have detected an anomaly in Service X, what is the correct response?

Condition Detected	Decision	Action Executed
Service returns no /health response	Service is down — restart it	docker restart <service-name>
CPU > 85% for 3+ consecutive readings	Service under load — simulate scale	Start additional container instance
Memory growth trend detected by ML	Potential memory leak — recycle service	docker restart <service-name>

Condition Detected	Decision	Action Executed
Response time > 2000ms consistently	Service degraded — restart	docker restart <service-name>
Error rate > 10% in log analysis	Application error — restart and alert	docker restart + dashboard alert

11.3 The Self-Healing Engine

The Self-Healing Engine is implemented using the Docker Engine API — a Unix socket interface that allows any program with the right permissions to send commands directly to the Docker daemon (the Docker background process). This means a Python script can call:

```
import docker
client = docker.from_env()
client.containers.get('order-service').restart()
```

And Docker will stop and restart the container, just as if the engineer had manually typed 'docker restart order-service' in the terminal. The difference is ARES does this automatically, in under 2 seconds, at any time of day or night.

12. Helios Integration — The ML Engine in ARES

12.1 The Isolation Forest Model

The machine learning model at the heart of both Helios and the ARES ML Engine is called Isolation Forest. It is an unsupervised anomaly detection algorithm — meaning it does not need labeled training data (no need for manually tagged examples of 'normal' and 'abnormal' behavior).

Isolation Forest works by building many decision trees on the data. The core insight is: anomalous data points are easier to isolate than normal data points. A normal CPU reading of 45% sits in a dense cluster with thousands of similar readings — isolating it requires many splits. An anomalous reading of 95% is unusual — it gets isolated in very few splits. The fewer splits needed, the higher the anomaly score.

12.2 What the Model Learns

The model is trained on historical metric data collected from the running microservices. It learns what normal looks like for each service — normal CPU range, normal memory consumption, normal response time distribution. Once trained, it can detect deviations from this baseline in real-time.

Metric Fed to Model	Normal Range (Example)	Anomaly Trigger
CPU Usage	20% — 60% under regular load	Score > 0.7 when CPU sustained above 85%
Memory Consumption	200MB — 400MB steady state	Score > 0.7 when memory grows continuously
API Response Time	50ms — 300ms for typical requests	Score > 0.7 when latency exceeds 1500ms
Error Rate	0% — 2% normal operation	Score > 0.7 when errors exceed 8% of requests

12.3 The Integration Architecture

In ARES, the Helios Flask API runs as a Docker container alongside the microservices. The Monitoring Agent collects metrics from Prometheus, then POSTs them to the Helios ML API endpoint. The ML API returns an anomaly score between 0 and 1. If the score exceeds the configured threshold (default: 0.7), the Monitoring Agent notifies the Decision Engine, which triggers the Healing Agent.

```
Prometheus scrapes metrics  ->  Monitoring Agent collects data
```

```
Monitoring Agent  ->  POST /predict/cpu {"value": 91}  ->  Helios ML
API
Helios ML API  ->  {"anomaly": true, "score": 0.84}
Monitoring Agent  ->  notifies Decision Engine
Decision Engine  ->  triggers Healing Agent
Healing Agent  ->  docker restart order-service
```

13. The Self-Healing Demo — Exactly What Will Happen

13.1 The Demo Scenario

This is the exact sequence of events that will be demonstrated during the Phase 2 presentation. Every step is defined, measurable, and reproducible.

Step	Actor	What Happens	Visible Evidence
1	User / Evaluator	Opens the e-commerce website and browses products normally	Website loads, products visible
2	Load Agent	Generates high traffic — simulates 200 concurrent users hitting the Order Service	Grafana shows request rate spike
3	System	Order Service container CPU spikes to 90%+, response time increases to 3000ms+	Grafana graphs show the spike in real time
4	Monitoring Agent	Detects the metric spike in Prometheus data, sends values to Helios ML API	Log shows 'Anomaly detected: order-service CPU 91%'
5	Helios ML Engine	Returns anomaly score 0.87 — above threshold	Log shows 'Score: 0.87 — ANOMALY'
6	Decision Engine	Determines action: restart order-service container	Log shows 'Decision: restart order-service'
7	Healing Agent	Executes docker restart order-service	Log shows 'Healing action executed'
8	System	Order Service restarts in 2-4 seconds, CPU returns to normal	Grafana shows CPU drop, service back online
9	ARES Dashboard	Recovery event appears in dashboard with full timeline	Dashboard shows full incident lifecycle
10	User	Website continues to work normally — zero downtime from user perspective	Website still accessible throughout

Demo Impact	The evaluator watches a service fail under load — and then watches the system fix itself with no human intervention. This is the entire value proposition of ARES demonstrated in under 2 minutes.

14. Team Structure and Responsibilities

Team Member	Role	Phase 1 Contribution	Phase 2 Responsibility
Nitanshu Tak	System Architect & Reliability Lead	Backend microservices architecture, API design, database setup, deployment coordination, project documentation	ARES Control Plane, Docker setup, Agent development, ML Engine integration (Helios), AWS deployment
Aarohi Jaiswal	Frontend Developer & UI/UX Designer	Complete Next.js frontend design and implementation, all pages, user flows, responsive design, visual identity	Frontend ARES Dashboard — service health UI, ML alert display, recovery event timeline
Aakriti Dhawan	Cloud Infrastructure Engineer	Backend service deployment on Render, Neon database setup, environment configuration	AWS EC2 setup, security groups, VPC, Prometheus installation, Grafana dashboard configuration
Sristee	DevOps & CI/CD Engineer	GitHub repository management, code integration support	GitHub Actions CI/CD pipeline, Docker image builds, Amazon ECR push, logging stack (ELK/Loki)

14.1 AWS IAM User Structure

For Phase 2, two AWS IAM (Identity and Access Management) users will be created to give cloud access to the team members responsible for infrastructure:

- **ares-cloud-admin (Aakriti) —** EC2 Full Access, VPC Access, CloudWatch, ECR Read Access. Responsible for the cloud infrastructure layer.
- **ares-devops (Sristee) —** ECR Full Access, EC2 Read/Deploy, CloudWatch Logs, GitHub Actions integration. Responsible for the automation and deployment pipeline.

15. Glossary — Every Technical Term Explained

This section defines every technical term used in this document in plain, simple language.

Term	Plain English Explanation
Microservices	Instead of one big program that does everything, you build many small programs (services) each doing one thing. Like separate departments in a company.
Docker	A tool that packages a program and everything it needs to run into a self-contained box called a container. Any computer with Docker can run the container identically.
Container	A running instance of a Docker image. Like a virtual mini-computer with its own process, file system, and network, but sharing the host machine's OS kernel.
API	Application Programming Interface — a defined way for two programs to talk to each other. Like a waiter in a restaurant: you give the order (request), they bring the food (response).
REST API	A specific style of API that uses standard HTTP methods (GET, POST, DELETE) and URLs. The most common way web services communicate.
JWT	JSON Web Token — a small encrypted string that proves who you are. Like a digital ID card that the server can verify without calling a database.
PostgreSQL	A database system that stores data in structured tables with rows and columns. Used when data relationships and transactions are important.
Prometheus	A monitoring system that regularly calls a /metrics endpoint on each service and stores the numbers it gets back as time-series data.
Grafana	A dashboard tool that connects to Prometheus and turns the stored metrics into real-time graphs and charts.
Isolation Forest	A machine learning algorithm that detects anomalies by finding data points that are unusually easy to separate from the rest of the data.
Anomaly Detection	Using algorithms to find data points that are unusually different from normal patterns — in ARES, this means detecting unusual CPU, memory, or latency readings.
CI/CD	Continuous Integration / Continuous Deployment — the practice of automatically building and deploying code every time a change is pushed to the repository.
MAPE-K	Monitor, Analyze, Plan, Execute, Knowledge — a framework from computer science describing how autonomous systems manage themselves.
AWS EC2	Amazon Web Services Elastic Compute Cloud — virtual machines (servers) in the cloud. You rent computing power by the hour.
AWS IAM	Identity and Access Management — AWS's system for controlling who can access what resources in the cloud.

Term	Plain English Explanation
ELK Stack	Elasticsearch, Logstash, Kibana — a set of tools for collecting, storing, and searching log files. Used for log-based anomaly detection.
Health Endpoint	A URL that a service exposes (e.g. GET /health) that returns OK if the service is running correctly. ARES agents check this to know if a service is alive.
Bcrypt	A password hashing algorithm. Transforms a plain text password into a long string that cannot be reversed — so passwords are never stored in readable form.
Neon	A cloud-hosted PostgreSQL service with a free tier. Your database lives on Neon's servers, not on your own computer.
Vercel	A cloud platform for deploying Next.js applications. Connects to GitHub and auto-deploys every time you push new code.
Render	A cloud platform for deploying web services. Similar to Heroku — push your code, get a live URL. Used for the 4 backend microservices.
DevSecOps	Development + Security + Operations — the practice of integrating security into the DevOps workflow. The research area of the Helios paper.
Self-Healing	The ability of a system to detect its own failures and automatically execute recovery actions without human intervention.
SRE	Site Reliability Engineering — a discipline invented at Google that applies software engineering principles to maintaining reliable production systems.

ARES — Autonomous Reliability Engineering System

Phase 1 Complete • Phase 2 In Progress

Built on Helios — Accepted Conference Research in DevSecOps & AI/ML Cloud Anomaly Detection

GitHub: github.com/Nitanshu715/ARES-Autonomous-Reliability-Engineering-System